

一、集成学习方法的数学描述

1、Bagging (bootstrap aggregating)

- **核心思想**：基于自助采样 (bootstrap) 生成多个独立训练集，训练多个基模型后融合。
- **数学描述**：

1. 从原始训练集 $D = \{(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)\}$ 中，通过有放回采样生成 M 个大小为 N 的子集 $D_1, D_2, \dots, D_M$ 。
2. 用每个子集训练一个基模型 $h_m(x)$ (基模型结构相同，训练独立)。
3. 预测时，分类问题采用**多数投票** $H(x) = \operatorname{argmax}(\sum_{m=1}^M I(h_m(x) = y))$ ， I 为指示函数。回归问题采用**均值融合** $H(x) = \frac{1}{M} \sum_{m=1}^M h_m(x)$ 。

2、Boosting (以 AdaBoost 为例)

- **核心思想**：迭代训练基模型，每次调整样本权重 (错分样本权重升高) 和模型权重 (性能好的模型权重升高)，最终加权融合。
- **数学描述**：

1. 初始化样本权重 $w_{1i} = \frac{1}{N}$ ($N = 1, 2, \dots, N$) 迭代次数为 M 。
2. 第 m 轮：用带权重的训练集 (D, w_m) 训练基模型 $h_m(x)$ ，计算模型误差率 $\epsilon_m = \sum_{i=1}^N w_{mi} I(h_m(x_i) \neq y_i)$ 。
3. 计算基模型权重 $\alpha_m = \frac{1}{2} \ln(\frac{1-\epsilon_m}{\epsilon_m})$ ($\epsilon_m < 0.5$) 时，误差越小 α_m 越大)。
4. 更新样本权重 $w_{(m+1)i} = \frac{w_{mi}}{Z_m} \exp(-\alpha_m y_i h_m(x_i))$ ，(Z_m 为归一化因子，保证权重和为 1)。
5. 最终融合模型 $H(x) = \operatorname{sign}(\sum_{m=1}^M \alpha_m h_m(x))$ (分类问题)，回归问题类似加权求和。

3、Stacking (堆叠集成)

- **核心思想**：用初级基模型的预测结果作为次级模型的输入，训练次级模型完成最终预测。
- **数学描述**：

1. 设初级基模型集合为 $\{h_1(x), h_2(x), \dots, h_K(x)\}$ ，次级模型为 $G(z)$ 。
2. 将原始训练集 D 划分为 S_1 和 S_2 ，用 S_1 训练初级模型，用 S_2 得到初级预测结果 $z_i = (h_1(x_i), h_2(x_i), \dots, h_K(x_i))$ ($x_i \in S_2$)。
3. 构建次级训练集 $D' = \{(z_i, y_i) \mid x_i \in S_2\}$ ，训练次级模型 $G(z)$ 。
4. 预测时，先通过所有初级模型得到 $z = (h_1(x), h_2(x), \dots, h_K(x))$ ，再输入 $G(z)$ 得到最终结果 $H(x) = G(z)$ 。

二、Bagging 机制、算法和特点

1、核心定义

Bagging 是“自助采样集成” (Bootstrap Aggregating) 的缩写，核心逻辑是通过**随机采样生成独立训练集、并行训练多个基模型、简单融合基模型预测结果**，最终降低单一模型的方差（减少过拟合），提升预测的稳定性和泛化能力。

2、完整实现流程

设原始训练集为 $D = \{(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)\}$ ，其中 N 为样本数， $x_i \in \mathbb{R}^d$ (d 为特征维度)， y_i 为标签（分类任务 $y_i \in \mathcal{Y}$ ，回归任务 $y_i \in \mathbb{R}$ ，基模型集合为 $\{h_1, h_2, \dots, h_M\}$ (M 为基模型个数，通常为同质模型)，具体步骤如下：

1. **自助采样 (Bootstrap Sampling)**：从原始数据集 D 中，通过**有放回随机采样**生成 M 个独立的训练子集 $D_1, D_2, \dots, D_M$ ，每个子集大小与原始数据集一致（均为 N ）。

（注：每个样本被选入单个子集的概率约为 $1 - \frac{1}{e} \approx 63.2\%$ ，未被选入的样本称为“袋外样本”，可用于无额外数据的模型验证）。

2. **并行训练基模型**：对每个子集 D_m ($m = 1, 2, \dots, M$)，独立训练一个基模型 $h_m(x)$ 。

基模型训练过程互不干扰（并行执行），且基模型结构相同（如同质决策树、同质神经网络等）。

3. **融合预测结果**：

- 分类任务：采用**多数投票法**（硬投票）或**概率加权投票法**（软投票）。

硬投票： $H(x) = \operatorname{argmax}(\sum_{m=1}^M I(h_m(x) = y))$ ($I(\cdot)$ 为指示函数，满足条件则为 1，否则为 0)；

软投票： $H(x) = \operatorname{argmax}(\sum_{m=1}^M P_m(y|x))$ ($P_m(y|x)$ 为第 m 个基模型预测 x 属于类别 y 的概率)。

。 回归任务：采用均值融合或中位数融合。

均值融合： $H(x) = \frac{1}{M} \sum_{m=1}^M h_m(x)$ ；

中位数融合： $H(x) = \operatorname{median}\{h_1(x), h_2(x), \dots, h_M(x)\}$ (抗异常值能力更强)。

3、Bagging 的代表性算法

1. **随机森林 (Random Forest, RF)**：最经典的 Bagging 改进算法，核心是在 Bagging 基础上增加“特征随机选择”：

每个基决策树训练时，从 d 个特征中随机选择 k 个特征 (通常 $k = \sqrt{d}$)，仅基于这 k 个特征进行分裂。

该改进进一步降低了基模型间的相关性，提升了集成效果，是目前应用最广泛的集成算法之一。

2. **Extra-Trees (极端随机树)**：RF 的变体，在“特征随机选择”基础上，进一步随机化决策树的分裂阈值 (而非基于信息增益 / 基尼系数选择最优阈值)，训练速度更快，方差更低。
3. **基于 Bagging 的神经网络集成**：用 Bagging 采样生成多个训练集，训练结构相同的神经网络，最终通过投票 / 均值融合预测结果，降低神经网络的过拟合风险。

4、Bagging 机制特点的数学语言描述

1. 基模型的独立性与低相关性

设 $h_m(x)$ 和 $h_l(x)$ ($m \neq l$) 为任意两个基模型，其预测误差的协方差满足：

$$\operatorname{Cov}(h_m(x), h_l(x)) = \mathbb{E}[h_m(x)h_l(x)] - \mathbb{E}[h_m(x)]\mathbb{E}[h_l(x)]。$$

由于 Bagging 采用独立自助采样，且基模型并行训练无交互，使得 $\operatorname{Cov}(h_m(x), h_l(x))$ 显著降低 (甚至接近 0)。核心结论：**基模型间低相关性是**

Bagging 降低方差的关键——集成模型的方差 $Var(H(x)) = \frac{1}{M} Var(h_m(x)) + \frac{M-1}{M} Cov(h_m(x), h_l(x))$ ，当 Cov 减小时， $Var(H(x))$ 随之降低。

2. 集成模型的偏差近似不变

设单个基模型的偏差为 $Bias(h_m(x)) = \mathbb{E}[h_m(x)] - f(x)$ ($f(x)$ 为真实目标函数)，则集成模型的偏差为：

$$\begin{aligned} Bias(H(x)) &= \mathbb{E}[h_m(x)] - f(x) = \mathbb{E}\left[\frac{1}{M} \sum_{m=1}^M h_m(x)\right] - f(x) \\ &= \frac{1}{M} \sum_{m=1}^M \mathbb{E}[h_m(x)] - f(x) = Bias(h_m(x)) \end{aligned}$$

(因所有基模型同质且独立采样， $\mathbb{E}[h_m(x)]$ 均相等)。核心结论：**Bagging 不改变模型的偏差，仅降低方差**——适用于偏差小、方差大的基模型（如决策树、神经网络），对偏差大的基模型（如线性回归）提升有限。

3. 预测结果的渐近一致性

当基模型个数 $M \rightarrow \infty$ 时，集成模型的预测结果满足大数定律：

- 回归任务： $H(x) = \frac{1}{M} \sum_{m=1}^M h_m(x) \xrightarrow{P} \mathbb{E}[h_m(x)]$ （依概率收敛到基模型的期望预测值）；
- 分类任务（硬投票）：若单个基模型的预测准确率 $p > 0.5$ ，则 $H(x) = \operatorname{argmax}(\sum_{m=1}^M I(h_m(x) = y)) \xrightarrow{P} f(x)$ （依概率收敛到真实标签）。

核心结论：随着基模型个数增加，**Bagging 集成模型的预测结果逐渐稳定，泛化误差趋于收敛（不会因模型过多而退化）**。

4. 袋外误差的无偏估计性

设 $D_{OOB}(x_i)$ 为样本 x_i 未被选入的基模型集合（袋外模型集），其个数为 $M_{OOB}(x_i)$ ，则袋外误差为： $Err_{OOB} = \frac{1}{N} \sum_{i=1}^N L(y_i, \frac{1}{M_{OOB}(x_i)} \sum_{h_m \in D_{OOB}(x_i)} h_m(x_i))$ ($L(\cdot, \cdot)$ 为损失函数，如分类任务的 0-1 损失、回归任务的平方损失)。数学证明： Err_{OOB} 是泛化误差 $Err_{Gen} = \mathbb{E}[L(y, H(x))]$ 的无偏估计，即 $\mathbb{E}[Err_{OOB}] = \mathbb{E}[Err_{Gen}]$ 。
核心结论：Bagging 无需额外验证集，可通过袋外误差直接评估模型泛化能力，简化模型选择流程。

三、Boosting 机制、算法和特点

1、Boosting 机制

Boosting 是一类迭代式集成学习框架，核心逻辑是：通过逐步训练基模型，每次聚焦前序模型的错误样本（调整样本权重），同时给性能更优的基模型分配更高权重，最终将多个“弱基模型”加权融合为“强集成模型”，核心目标是降低模型偏差（提升拟合能力），同时控制方差。

2、完整实现流程

设原始训练集 $D = \{(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)\}$ (N 为样本数, $x_i \in \mathbb{R}^d$, y_i 为标签), 基模型集合 $\{h_1, h_2, \dots, h_M\}$ (M 为迭代次数 / 基模型个数, 通常为同质弱模型, 如单层决策树、线性模型), 损失函数 $L(h(x), y)$, 具体步骤如下:

1. 初始化样本权重: 给所有样本分配相等的初始权重 $w_{1i} = \frac{1}{N}$ ($i = 1, 2, \dots, N$), 保证初始训练集分布均匀。
2. 迭代训练基模型: 对第 m 轮 ($m = 1, 2, \dots, M$):
 - 基于带权重的训练集 (D, w_m) 训练基模型 $h_m(x)$ (基模型学习目标是 最小化加权损失 $\sum_{i=1}^N w_{mi} L(y_i, h_m(x_i))$);
 - 计算基模型 h_m 的加权误差率 $\epsilon_m = \frac{\sum_{i=1}^N w_{mi} I(h_m(x_i) \neq y_i)}{\sum_{i=1}^N w_{mi}}$ (分类任务, $I(\cdot)$ 为指示函数), 或 $\epsilon_m = \sum_{i=1}^N w_{mi} L(y_i, h_m(x_i))$ (回归任务);
 - 计算基模型权重 α_m : 权重与模型性能正相关, 如分类任务 $\alpha_m = \frac{1}{2} \ln \left(\frac{1-\epsilon_m}{\epsilon_m} \right)$ ($\epsilon_m < 0.5$) 时, 误差越小, α_m 越大; 若 $\alpha_m \geq 0.5$, 则丢弃该模型; 回归任务可设 $\alpha_m = \frac{1}{\sqrt{2\pi}\sigma_m} \exp \left(-\frac{\epsilon_m^2}{2\sigma_m^2} \right)$ (σ_m 为 h_m 预测误差的标准差);
 - 更新样本权重 $w_{(m+1)i}$: 对错误预测的样本升高权重, 正确预测的样本降低权重, 再归一化保证权重和为 1。分类任务更新公式:

$$w_{(m+1)i} = \frac{w_{mi} \exp(-\alpha_m y_i h_m(x_i))}{Z_m}$$

($Z_m = \sum_{i=1}^N w_{mi} \exp(-\alpha_m y_i h_m(x_i))$ 为归一化因子, $y_i \in \{-1, 1\}$);

回归任务可设 $w_{(m+1)i} = \frac{w_{mi} \exp(\alpha_m |y_i - h_m(x_i)|)}{Z_m}$ 。

3. 加权融合预测：

- 分类任务： $H(x) = \text{sign}(\sum_{m=1}^M \alpha_m h_m(x))$ (符号函数输出最终类别)；
- 回归任务： $H(x) = \sum_{m=1}^M \alpha_m h_m(x) / \sum_{m=1}^M \alpha_m$ (加权均值，保证权重归一化)。

2、Boosting 的代表性算法

1. AdaBoost (自适应提升)：最经典的 Boosting 算法，适用于二分类任务，采用 0-1 损失函数，通过样本权重和模型权重的自适应调整实现提升，核心是“聚焦错分样本”。
2. Gradient Boosting (梯度提升)：将 Boosting 过程转化为“梯度下降优化”，核心是每次训练一个基模型来拟合前序集成模型的损失函数负梯度（替代 AdaBoost 的样本权重调整），支持任意可微损失函数，灵活性更强。
3. XGBoost (极端梯度提升)：Gradient Boosting 的工程化改进版，加入正则化项 (L1/L2 正则)、缺失值处理、并行特征选择等，同时通过二阶泰勒展开加速损失函数优化，泛化能力和训练速度显著提升。
4. LightGBM / CatBoost：基于 Gradient Boosting 的高效实现，分别通过“直方图优化”“类别特征编码优化”等降低计算复杂度，支持大规模数据训练，且对类别特征、不平衡数据有更好的适配性。